Lecture 2: Introduction to Machine Learning
Part 2

Md. Shahriar Hussain
ECE Department, NSU

Types of Machine Learning Systems

- There are so many different types of Machine Learning systems that it is useful to classify them in broad categories, based on the following criteria:

1. Does it require human supervision?
   - Supervised Learning
   - Semisupervised Learning
   - Unsupervised Learning
   - Reinforcement Learning

Types of Machine Learning Systems

2. Can it learn incrementally on the fly?
   - Online Learning
   - Batch Learning

3. Does the system build a predictive model?
   - Model-based Learning
   - Instance-based Learning

Batch Learning vs Online Learning

- Batch Learning
  - Not capable of learning after deployment
  - Must be retrained from scratch – computationally expensive!
  - it is typically done offline
  - If you want a batch learning system to know about new data you need to train a new version of the system from scratch on the full dataset

- Online Learning
  - Can continue to learn after deployment
  - Can take advantage of parallel computing – no down time
  - In *online learning*, you train the system incrementally by feeding it data instances sequentially, either individually or in small groups called *mini-batches*
  - Online learning is great for systems that receive data as a continuous flow (e.g., stock prices) and need to adapt to change rapidly or autonomously.
  - It is also a good option if we have limited computing resources:
  - once an online learning system has learned about new data instances, it does not need them anymore, so we can discard them

Online Learning

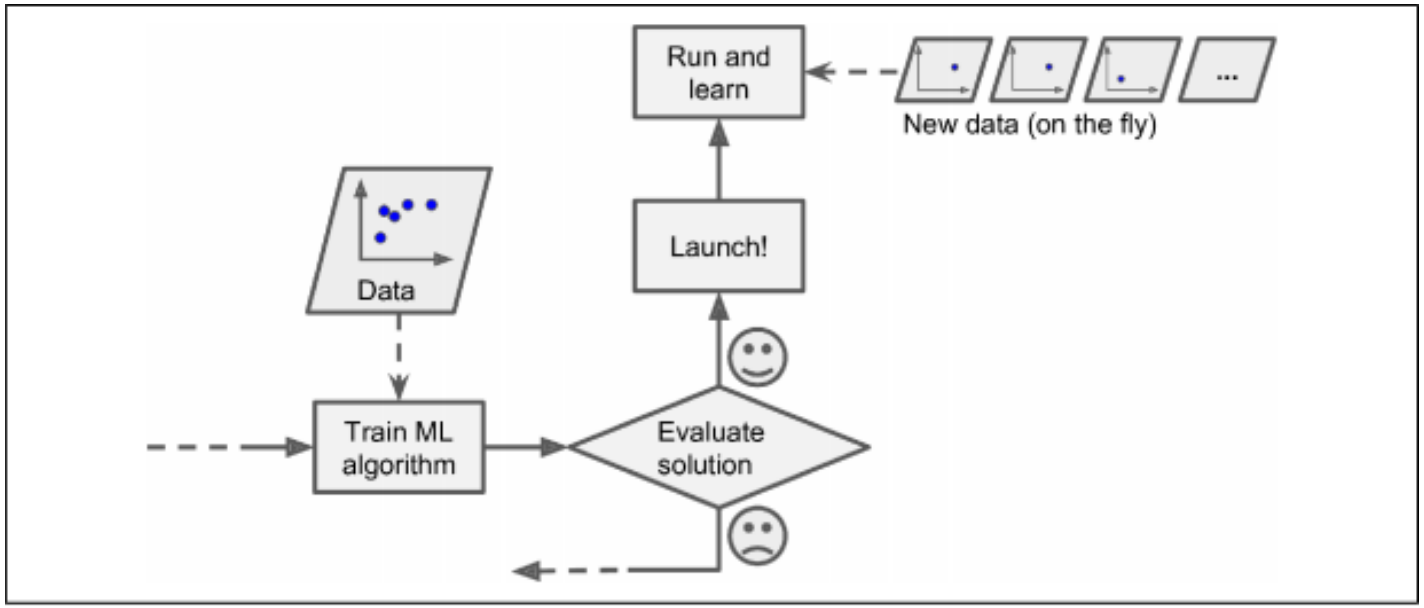


Figure 1-13. In online learning, a model is trained and launched into production, and then it keeps learning as new data comes in

Instance-Based vs Model-Based Learning

- Instance-based Learning
  - Memorize known data
  - Use similarity measure to generalize new instances
  - e.g. new instance is a triangle because it's similar to the other triangles

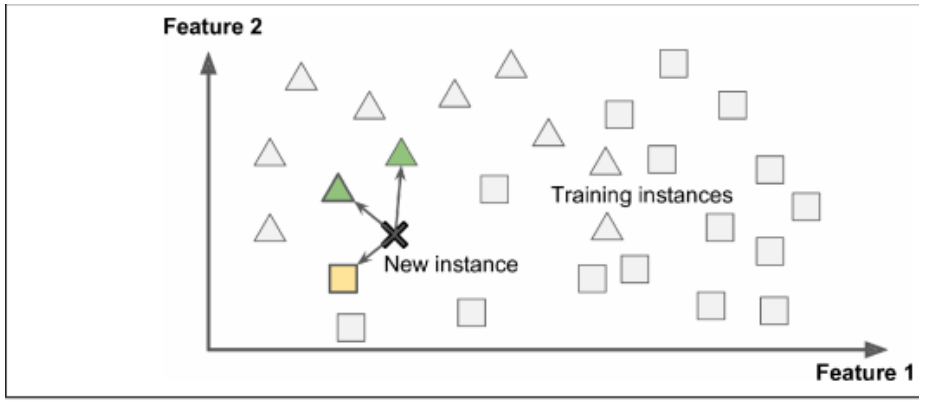


Figure 1-15. Instance-based learning

- Model-based Learning
  - Build model from training data
  - Predict based on model output

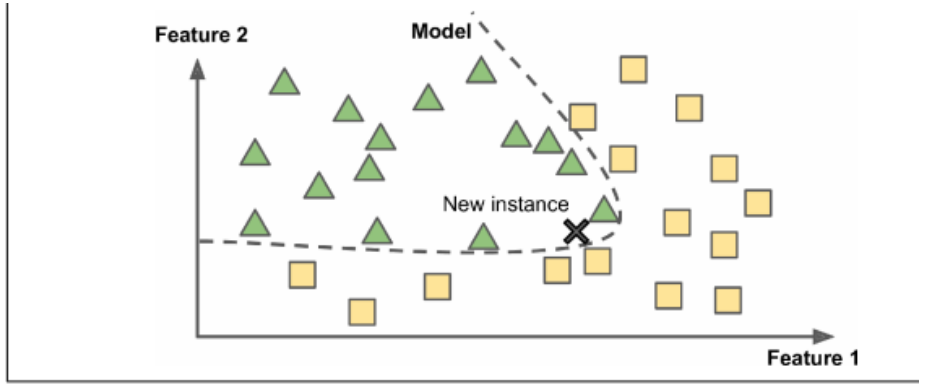


Figure 1-16. Model-based learning

Regression Problem

- **Regression algorithms** are used when outputs have continuous values i.e. they may have any value within the range

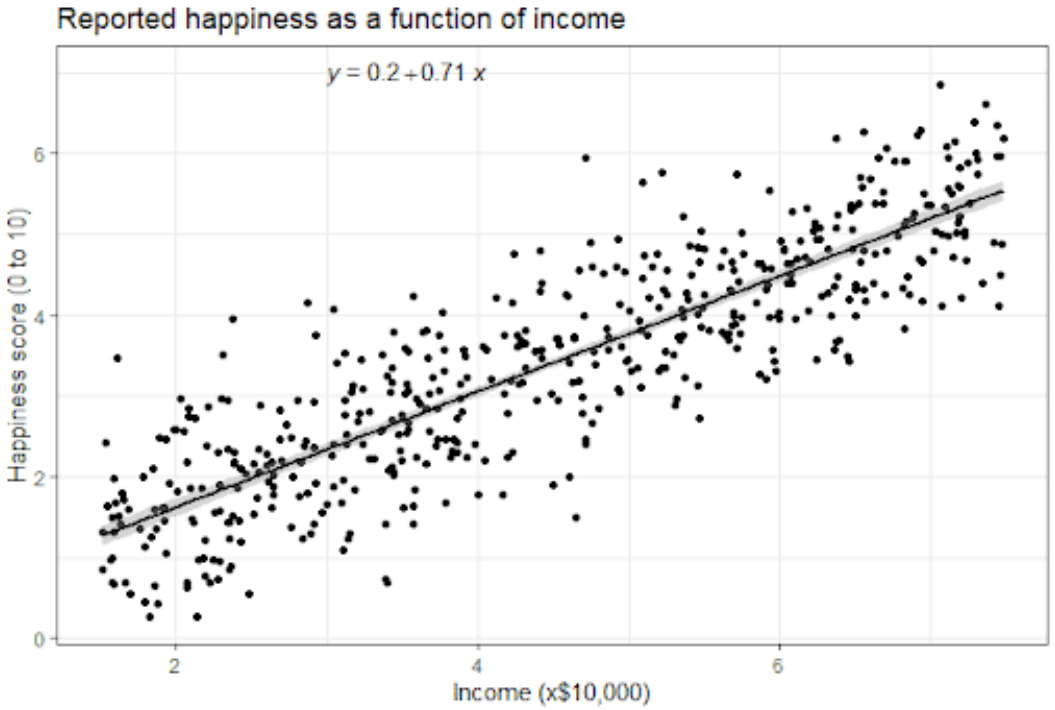
- The table is containing home prices in Monroe twp, NJ. Here price depends on area (square feet), bedrooms and age of the home (in years). Given these prices we have to predict prices of new homes based on area, bedrooms and age.

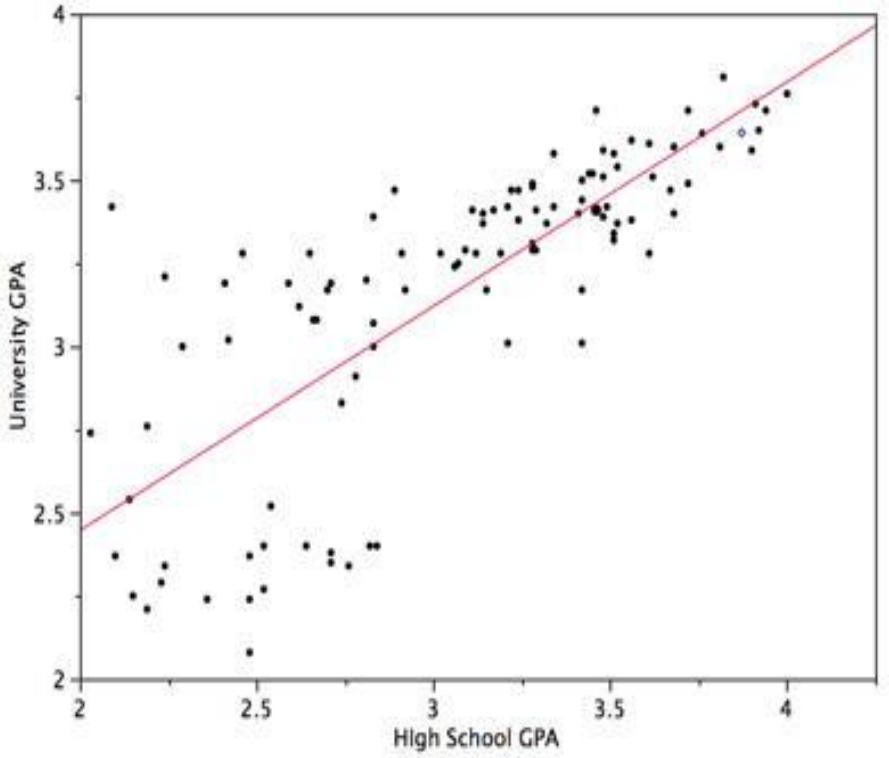
area	bedrooms	age	price
2600	3	20	550000
3000	4	15	565000
3200		18	610000
3600	3	30	595000
4000	5	8	760000
4100	6	8	810000

- Given these home prices, find out price of a home that has,
  - 3000 sqr ft area, 3 bedrooms, 40 year old
  - 2500 sqr ft area, 4 bedrooms, 5 year old

- Regression problem: **continuous value output**

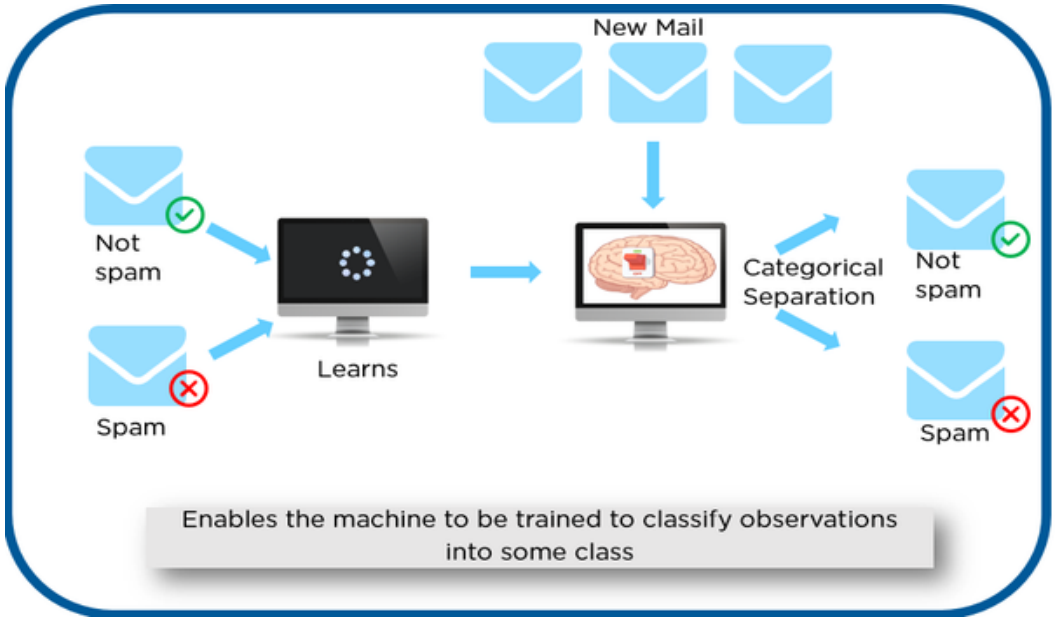
Example of Regression Problem





Classification Problem

- Classification algorithms are used when outputs are restricted to some discrete set of values
  - E.g. classifying whether an email is a spam email or not,
  - whether a tumor is malignant or benign
  - Classifying from medical records whether a pregnancy is high risk or not
  - Classify the species of iris https://en.wikipedia.org/wiki/Iris_flower_data_set



Example ML Task: Does money make people happy?

- Life Satisfaction data from OECD
- GDP per capita data from IMF

- What relationship can we infer between life satisfaction and GDP per capita from the graph?

Table 1-1. Does money make people happier?

Country	GDP per capita (USD)	Life satisfaction
Hungary	12,240	4.9
Korea	27,195	5.8
France	37,675	6.5
Australia	50,962	7.3
United States	55,805	7.2

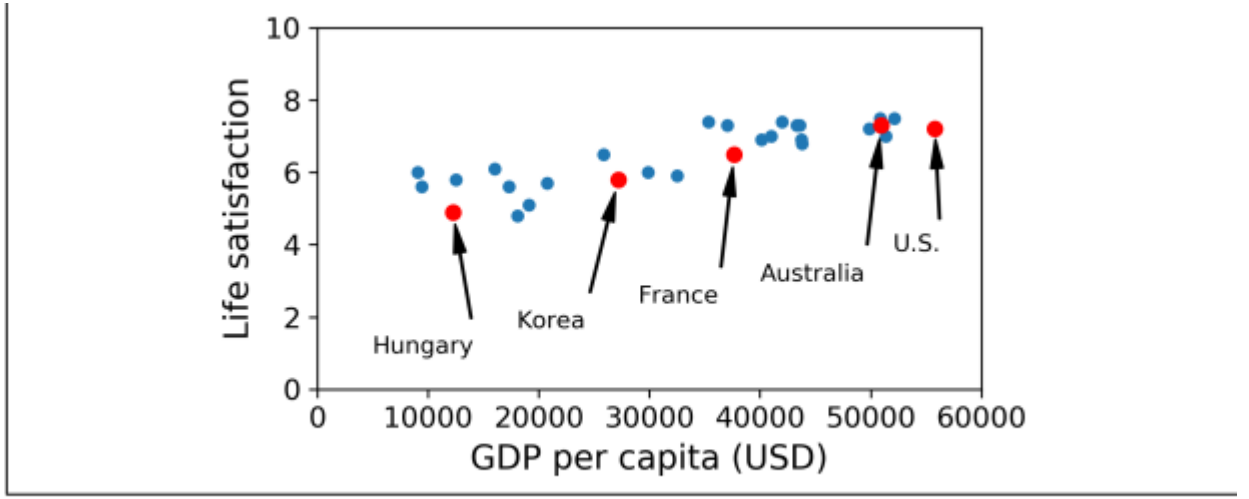


Figure 1-17. Do you see a trend here?

Model Selection

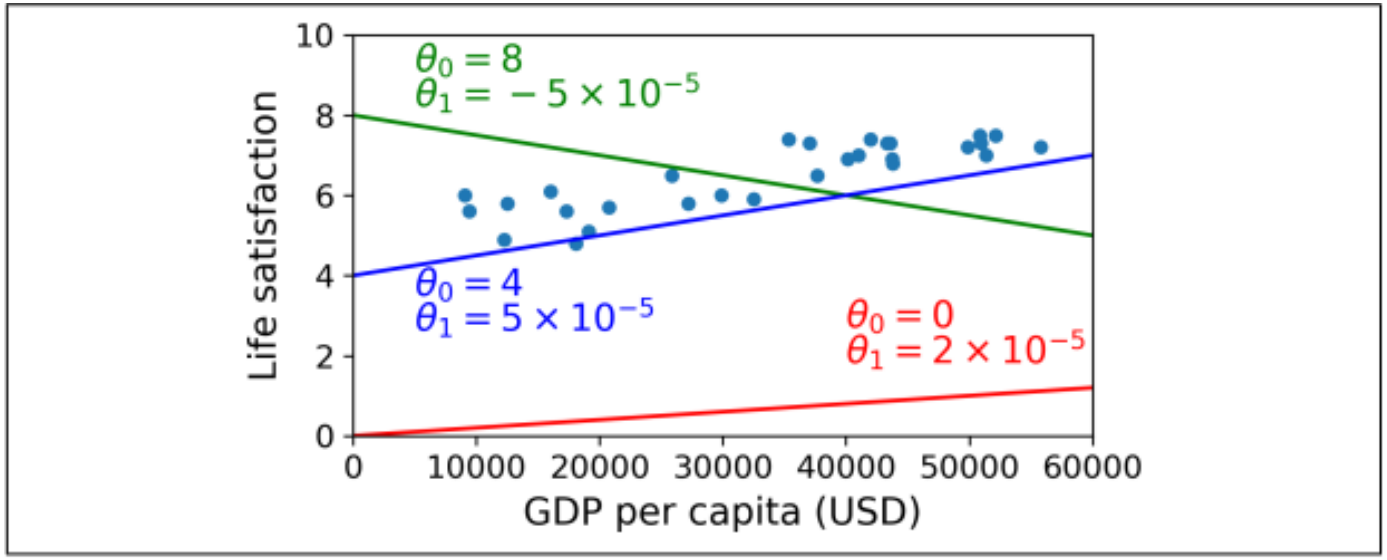


Figure 1-18. A few possible linear models

- Based on data, we can select a linear model of life satisfaction with just one feature/attribute, GDP per capita
- life_satisfaction = $\theta_0 + \theta_1 \times GDP\_per\_capita$
- This model has two *model parameters, y intercept* θ_0 *and slope* θ_1
- How can you know which values will make your model perform best?

Performance Measure

- Define a **utility** function (how good is the fitted line?), or a **cost** function (how bad is the fitted line?)
- **Linear Regression**
  - **Training**: try to minimize distance between linear model's prediction on the line, vs the actual training examples, until the estimated parameter values converge

- In our case, the algorithm finds that the optimal parameter values are $\theta_0 = 4.85$ *and* $\theta_1 = 4.91 \times 10–5$

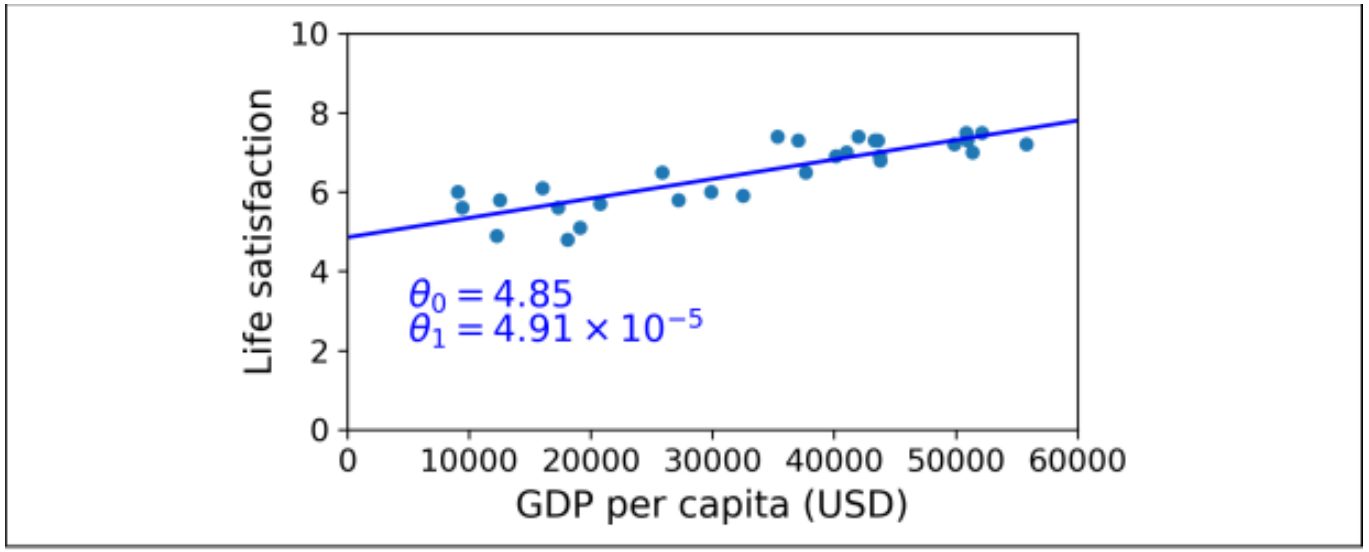
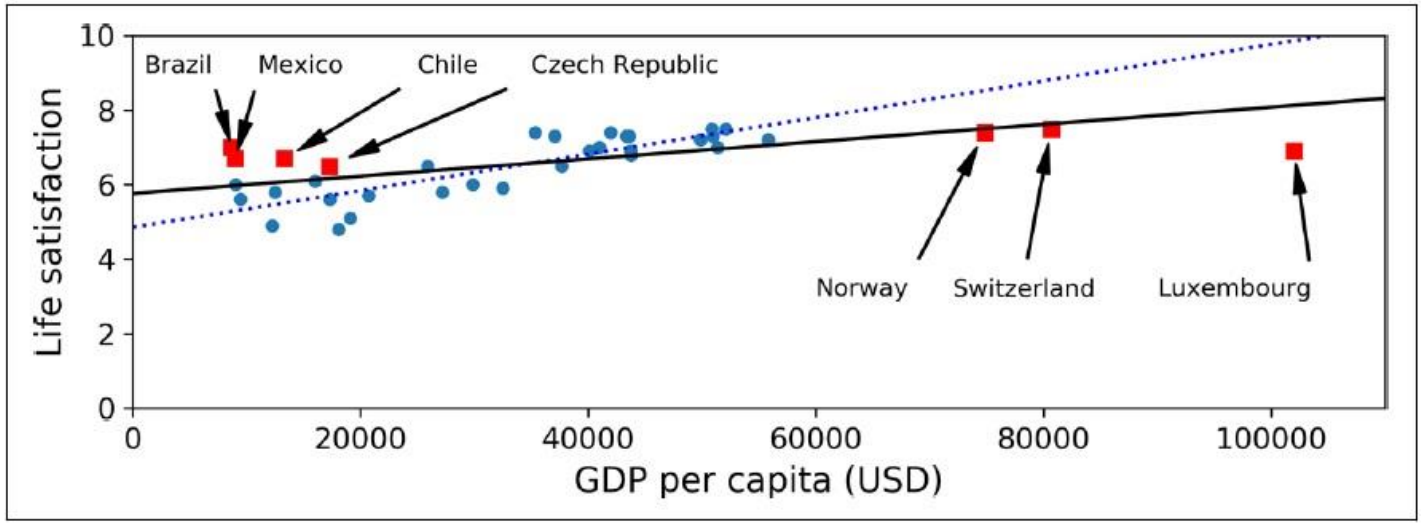


Figure 1-19. The linear model that fits the training data best

Problems with Machine Learning

- Problem #1: Training data
  - Insufficient quantity
  - Poor-quality data
  - Nonrepresentative data

Non representative data



- the set of countries we used earlier for training the linear model was not perfectly representative; a few countries were missing
- If we train a linear model on this data, we get the solid line, while the old model is represented by the dotted line.
- We can see, not only does adding a few missing countries significantly alter the model, but it makes it clear that such a simple linear model is probably never going to work well.

Problems with Machine Learning

- Problem #2: Which features should be used?
- A critical part of the success of a Machine Learning project is coming up with a good set of features to train on.
- This process, called *feature engineering, involves the* following steps:
  - *Feature selection (selecting the most useful features to train on among existing* features)
  - *Feature extraction (combining existing features to produce a more useful one*)
  - Creating new features by gathering new data

Problems with Machine Learning

- Problem #2: How “fit” is it?
  - Overfitting data
  - Underfitting data

Overfitting the Training Data

- Most common problem in ML – do not overgeneralize!
  - The polynomial model is better than the linear model on training
  - How about testing?

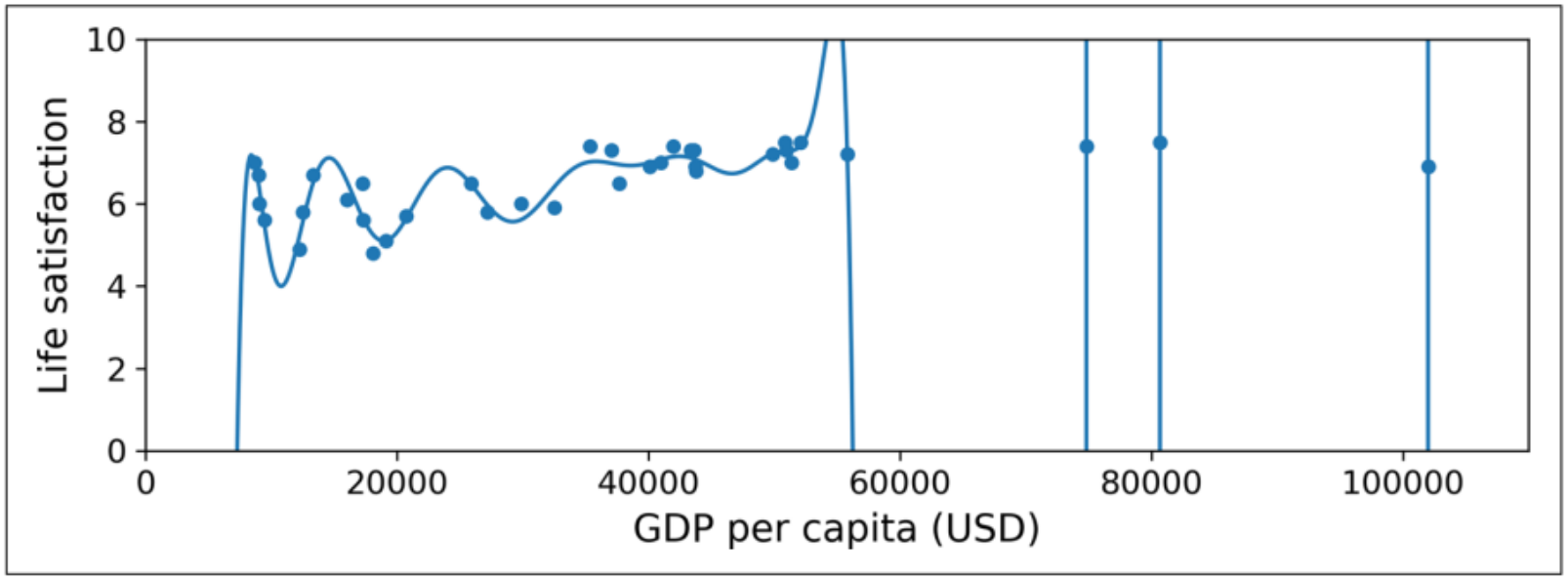


Figure 1-22. Overfitting the training data

How to avoid overfitting

- Tip #1: REGULARIZATION – USE IT
  - **Constrain** model to keep it simple – reduce risk of overfitting
  - **Hyperparameters –** control level of regularization
- Tip #2: Get more training data, and reduce noise in it

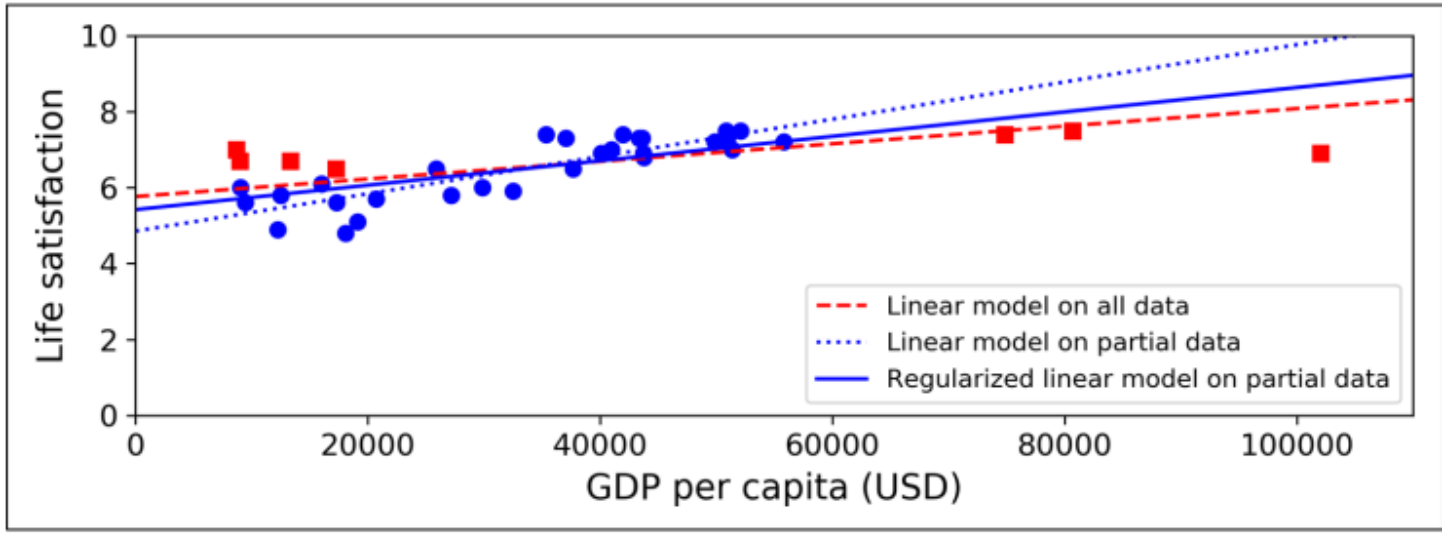


Figure 1-23. Regularization reduces the risk of overfitting

Underfitting the Training Data

- *Underfitting is the opposite of overfitting*
- *it occurs when your* model is too simple to learn the underlying structure of the data.
- For example, a linear model of life satisfaction is prone to underfit; reality is just more complex than the model,
- so its predictions are bound to be inaccurate, even on the training examples.

- Main options for fixing this problem:
  - Select a more powerful model, with more parameters.
  - Feed better features to the learning algorithm (feature engineering).
  - Reduce the constraints on the model (e.g., reduce the regularization hyperparameter).

Model Evaluation

- How good is your model?
  - Test it on **new data** – data not seen by the model ever before!
  - Keep **80%** for training, set **20%** for testing
  - NEVER go below 10% test data – better model is better than better “accuracy”

- How to regularize?
  - Keep a portion of **training data** held out for **validation**
  - Alternatively, use **cross-validation** (many validation sets instead of one)
  - Pick the hyperparameters that work best on validation for your model on the test dataset

Model Evaluation

- A great model
  - trained with 60% training data, 20% validation data, and 20% testing data
- An okay model
  - trained with 70% training data, 15% validation data, and 15% testing data
- A barely-acceptable model
  - trained with 80% training data, 10% validation data, and 10% testing data
- Models with worse ratios – hacks
  - Unless there's millions of instances in the dataset
- "No Free Lunch" theorem
  - Only way to know for sure which model works best is to evaluate them
  - Make reasonable assumptions about your data to select model